

SMART INDIA HACKATHON 2026

PROJECT REPORT

**DYNAMIC FORECAST OF EXPECTED TIME OF ARRIVAL
(ETA)
FOR COACHING TRAINS**

Problem Statement ID	SIH26028
Theme	Smart Automation
PS Category	Software
Team Name	CHAKRAx6
College / Institute	SKN Sinhgad College of Engineering
Team Members	Sanchita Lade Manish Nagare Vaishnavi Mane Raj Mharnur Raghavraj Kulkarni
Guide	Dr. V. G. Jagtap
Academic Year	2026-27

1. Executive Summary

SIH26028 focuses on dynamic forecasting of Expected Time of Arrival (ETA) for coaching trains. CHAKRAx6 proposes a self-correcting machine-learning engine that predicts ETAs network-wide by combining live railway data, section-wise forecasting, operational constraints, uncertainty estimation and continuous learning. The presentation defines the objective as reducing arrival-time error for passengers and controllers.

The proposed system fuses RTIS GPS, TMS, NTES and interlocking events into a unified stream. An Extended Kalman Filter (EKF) fills gaps between GPS pings. LightGBM and Temporal Convolutional Network (TCN) models forecast delay for each remaining section, while OR-Tools CP-SAT models headway and platform constraints to propagate delay effects. Each ETA includes a P10-P90 uncertainty band and a top explanatory cause using TreeSHAP. Re-forecasting is event-triggered, model versions are tracked with MLflow, and SimPy is used for digital-twin backtesting before deployment.

2. Problem Statement

Field	Details
Problem Statement ID	SIH26028
Problem Statement Title	Dynamic Forecast of Expected Time of Arrival (ETA) for Coaching Trains
Theme	Smart Automation
PS Category	Software
Team Name	CHAKRAx6

2.1 Problem and Need

Static or schedule-based arrival estimates do not fully capture changing network conditions. A coaching train may encounter congestion, signal restrictions, platform constraints, speed restrictions, weather effects or delays from trains ahead. The project therefore requires a continuously updated ETA that reacts to live conditions and also explains why the prediction changed.

3. Proposed Solution

The PPT proposes a self-correcting ML engine with seven core pillars:

- Live Data Fusion - RTIS GPS, TMS, NTES and interlocking events are merged into one stream; EKF fills gaps between GPS pings.
- Section-Wise Forecasting - LightGBM and TCN predict delay for each remaining section; sectional predictions are aggregated into downstream station ETAs.
- Delay Propagation - OR-Tools CP-SAT applies headway and platform constraints so one train's delay can influence trains behind it.
- Explainable ETA - every ETA carries a P10-P90 band and a top cause using TreeSHAP, such as fog or a speed restriction ahead.
- Event-Triggered Re-Forecast - only affected downstream ETAs are recomputed for low-latency response.
- Continuous Learning - nightly retraining is tracked in MLflow and each model version is benchmarked against the timetable baseline.
- Digital-Twin Backtesting - SimPy replays journeys to test accuracy before deployment.

4. Problem-Solution Comparison

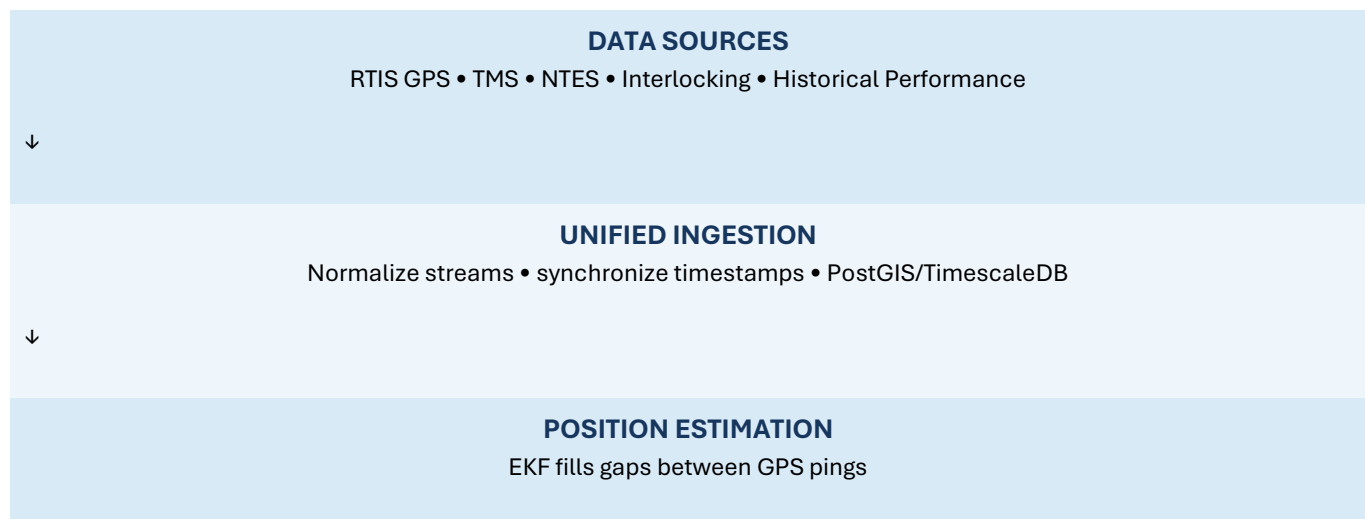
Area	Conventional / Static Approach	CHAKRAx6 Proposed Approach
Position updates	Individual or intermittent updates	Live fusion of RTIS GPS and operational streams with EKF gap filling
Forecasting	Schedule/current-delay based	Section-wise LightGBM + TCN forecasting
Network interaction	Limited delay propagation awareness	CP-SAT headway and platform constraints
Prediction output	Single ETA value	ETA plus P10-P90 uncertainty band
Explainability	Limited reason visibility	TreeSHAP top-cause explanation
Refresh strategy	Periodic/static	Event-triggered downstream re-forecast
Learning	Fixed rules/model	Nightly retraining with MLflow tracking
Validation	Historical/manual checks	SimPy digital-twin replay and benchmarking

5. Idea Title and Innovation

Self-correcting ML engine predicting coaching-train ETAs network-wide, cutting arrival-time error for passengers and controllers.

- Unified live-data fusion with dead-reckoning between GPS pings.
- Section-level forecasting rather than one delay value for the whole remaining journey.
- Constraint-aware delay propagation across trains.
- Prediction intervals and explainable causes for each ETA.
- Event-triggered low-latency recomputation.
- Continuous learning with model version tracking.
- Digital-twin validation before live deployment.

6. Process Flowchart





7. Detailed Technical Architecture

The technical approach in the PPT uses a modern web stack, a separate AI/optimization service, time-series/geospatial storage, event streaming and production monitoring.

Layer	Technologies from PPT	Role
Frontend	React, Vite, TypeScript, Tailwind, Wouter, Radix UI/shadcn, TanStack Query/Virtual, Recharts, D3.js, Canvas/Pixi.js	High-density train/network visualization, Marey graph and user interface

Layer	Technologies from PPT	Role
Backend	Node.js (Fastify/NestJS), tRPC + SuperJSON, Drizzle ORM, BullMQ/Redis, Redlock, Keycloak	Application APIs, jobs, locking and enterprise RBAC/OIDC
AI / Optimization	FastAPI, gRPC/Protobuf, OR-Tools CP-SAT, LightGBM, EKF, TCN, TreeSHAP, SimPy; PPO as stretch goal	ETA forecasting, delay propagation, explainability, simulation and disruption re-planning
Additional Models	lifelines Weibull survival model, PyTorch Autoencoder	Asset RUL and congestion anomaly detection as shown in the PPT architecture
Data	PostgreSQL + TimescaleDB + PostGIS, Redis 7, Kafka/Redpanda	Time-series/geospatial storage, caching/runtime state and telemetry streaming
Integrations	COA, TMS, SMMS, TDMS, iPAS, NTES, EI data loggers, axle counters, RTIS GPS gateway	Adapter layer with mock/replay sandboxes
DevOps	Docker Compose → Kubernetes, GitHub Actions, Vitest, pytest, Playwright, ESLint, MLflow, Prometheus, Grafana, OpenTelemetry	Deployment, CI/CD, testing, model tracking and observability

8. Data Sources and Integration

- RTIS GPS - live train location updates.
- TMS and NTES - train movement and information feeds identified in the PPT.
- Electronic Interlocking events and data loggers - operational events affecting movement.
- COA and timetable information - actual versus scheduled running context.
- Axle counters and other adapter-based railway sources - optional operational context.
- Historical performance - actual running times, sectional delays and congestion patterns.

The adapter layer is designed with configurable mock/replay sandboxes, allowing development and testing without affecting live railway operations.

9. AI/ML and Optimization Implementation

9.1 Extended Kalman Filter (EKF)

EKF is used for dead-reckoning and gap filling between GPS pings, maintaining a smoother estimate of train position when live location updates are intermittent.

9.2 Section-Wise ETA Forecasting

LightGBM provides risk/delay scoring and TCN handles temporal forecasting. Each remaining route section receives a delay or running-time estimate; the system then aggregates the sections to generate ETA at every downstream station.

9.3 Delay Propagation with CP-SAT

OR-Tools CP-SAT represents operational constraints such as headway and platform availability. This allows the forecast to account for how one delayed train can affect following trains rather than predicting each train independently.

9.4 Explainability and Confidence

TreeSHAP identifies the strongest reason for an ETA change. The prediction is presented with a P10-P90 interval so users can see forecast uncertainty instead of relying on a single exact-looking timestamp.

9.5 Continuous Learning and Digital Twin

The PPT specifies nightly retraining with MLflow model tracking and benchmarking against the timetable baseline. SimPy replays journeys as a digital twin so model behaviour can be backtested before deployment.

10. Technology Stack

Category	Selected Stack
Frontend	React, Vite, TypeScript, Tailwind, Wouter, Radix UI (shadcn/ui), TanStack Query, TanStack Virtual, Recharts, D3.js, Canvas/Pixi.js
Backend	Node.js (Fastify/NestJS), tRPC, SuperJSON, Drizzle ORM, BullMQ, Redis/Redlock, Keycloak
AI Service	FastAPI, gRPC/Protobuf
Forecasting	LightGBM, TCN, EKF
Optimization	OR-Tools CP-SAT; PPO stretch goal
Explainability	TreeSHAP and constraint conflict traces
Simulation	SimPy
Database	PostgreSQL, TimescaleDB, PostGIS
Streaming/Cache	Kafka/Redpanda, Redis 7
MLOps/Monitoring	MLflow, Prometheus, Grafana, OpenTelemetry
Testing/CI	GitHub Actions, Vitest, pytest, Playwright, ESLint
Deployment	Docker Compose → Kubernetes (hybrid on-prem/cloud)

11. Database Design / ER Model

The PPT does not provide a dedicated ER diagram, so the following report model is derived directly from the data flows and technologies shown in the technical architecture. It can be converted into a graphical ER diagram for the final submission.

Entity	Key Data
Train	train_id, train_number, train_type
Station	station_id, station_code, location

Entity	Key Data
Route Section	section_id, from_station, to_station, headway, restrictions
Timetable	train_id, station_id, scheduled_arrival, scheduled_departure
RTIS Position	train_id, timestamp, latitude, longitude, speed, source_quality
Operational Event	event_id, section_id, type, timestamp, severity
Historical Run	train_id, journey_date, section_id, actual_runtime, delay
ETA Prediction	train_id, target_station, generated_at, eta, p10, p90, top_cause, model_version
Model Version	model_version, training_date, benchmark_metrics
Feedback / Actual Arrival	train_id, station_id, actual_arrival, prediction_error

12. Feasibility and Viability

12.1 Technical Feasibility

- Uses proven technologies: Python, LightGBM, OR-Tools CP-SAT, SimPy and PostgreSQL/PostGIS.
- Designed to integrate with existing railway data sources and operational systems.
- Mock mode enables testing without affecting live railway operations.
- Modular architecture allows gradual deployment and scaling.

12.2 Operational Feasibility

- Supports existing control-room decision-making rather than replacing human controllers.
- Provides explainable ETA and block recommendations for approval.
- Handles real-time changes through continuous data updates and re-planning.
- Can be piloted on a single railway section/zone before wider deployment.

12.3 Economic Viability

- Reduces avoidable train-delay minutes and operational conflicts.
- Enables better utilization of maintenance windows and railway assets.
- Predictive maintenance can help reduce disruption from unexpected restrictions.
- Software-based architecture minimizes additional infrastructure requirements.

12.4 PPT Impact Distribution

Area	Share shown in PPT
Train ETA accuracy	27.8%
Predictive maintenance	22.2%
Delay reduction	22.2%
Network efficiency	16.7%
Environmental impact	11.1%

These percentages are reproduced from the feasibility slide and should be treated as presentation-stage impact weighting unless the team has a measured methodology behind them.

13. Impact and Benefits

The following targets and benefits are taken from the team PPT and should be validated during prototype testing before being presented as measured field performance:

- Real-time ETA updates every 30 seconds instead of static schedule-based estimates.
- Target ± 5 -8 minute prediction accuracy at intermediate and destination stations, compared in the PPT with ± 30 -45 minutes today.
- Proactive delay management using prediction intervals and confidence scores for every arrival.
- Reduced passenger uncertainty through mobile alerts, station displays and multilingual announcements.
- Target 15-20% improvement in station resource utilization through predictive platform allocation and crowd management.
- Congestion-aware forecasting that models downstream track occupancy and delay cascades.
- Continuous learning from actual arrivals so model accuracy can improve over time.
- Architecture intended to scale to 13,000+ trains simultaneously across Indian Railways zones via an adapter layer.
- Explainable AI using SHAP values so control-room staff can understand why an ETA changed.
- Decision-support integration with existing NTES/RTIS systems rather than replacing them.

14. Before-and-After Comparison

Parameter	Before / Existing	Proposed / Target from PPT
ETA refresh	Static or less dynamic estimate	Every 30 seconds / event-triggered
Arrival accuracy	PPT cites ± 30 -45 min	PPT target ± 5 -8 min
Confidence information	Not emphasized	P10-P90 prediction interval
Reason for ETA change	Limited	TreeSHAP top cause
Delay cascades	Limited network interaction	Congestion/headway/platform-aware propagation
Learning loop	Limited	Actual arrivals $\rightarrow$ retraining
Control-room support	ETA display	Explainable prediction + operational dashboard
Passenger communication	Basic status	Mobile alerts, station displays, multilingual announcements

Important: the numerical accuracy and resource-utilization values above are targets/claims stated in the supplied PPT. They should be replaced or supported with prototype test results before final evaluation.

15. Dashboard and User Interfaces

- Control Room Dashboard - congestion heatmap, cascade alerts, crew planning and explainable ETA changes.
- Passenger Interfaces - mobile application, station displays and multilingual alerts.
- Time-Distance Marey Graph - D3.js visualization for train movement over time and distance.

- High-Density Track Topology - Canvas/Pixi.js for scalable railway-network visualization.
- System KPI Dashboard - ETA accuracy, delay reduction, network efficiency, model health and latency.

16. Testing and Evaluation Plan

- Benchmark every model version against the timetable baseline.
- Use SimPy digital-twin replay on historical/recorded journeys before deployment.
- Measure ETA error by station and forecast horizon.
- Test P10-P90 interval coverage and calibration.
- Test event-triggered re-forecast latency.
- Test stale/missing RTIS GPS data and EKF fallback behaviour.
- Test CP-SAT behaviour under headway and platform conflicts.
- Use mock/replay sandboxes for integration tests without live operational impact.
- Track model versions and experiments in MLflow.

17. Website, Video and Digital Resources

Team Project Website: [Add deployed website URL]

Team Demo Video: [Add actual demo video URL]

GitHub Repository: [Add actual repository URL]

The report intentionally leaves these as placeholders because the supplied PPT does not contain the team's final website, demo-video or GitHub URLs.

18. Suggested Repository Structure

```
SIH26028-ETA/
├── frontend/
├── backend/
├── ai-service/
│   ├── ekf/
│   ├── lightgbm/
│   ├── tcn/
│   ├── explainability/
│   └── simulation/
├── integrations/
├── database/
├── tests/
├── infra/
├── docs/
└── README.md
```

19. Research and References

The supplied PPT lists the following references/resources:

- Predicting Trains Delays using a Two-level Machine Learning Approach.
- A hierarchical machine learning framework for aggregated multi-leg arrival time prediction and transfer-feasibility integration in regional rail corridors.

- Real-Time Train Information System (RTIS).
- Centre for Railway Information Systems (CRIS).
- SHAP - Explainable AI.

The PPT page provides reference titles but the parsed source does not expose the underlying hyperlink destinations. The final report should use the exact links from the original editable PPT or the team's source list rather than inventing URLs.

20. Future Scope

- Expand from a single section/zone pilot to network-wide deployment.
- Add richer weather and disruption feeds for context-aware forecasting.
- Improve network interaction modelling for complex delay cascades.
- Evaluate PPO-based disruption re-planning as the stretch goal identified in the PPT.
- Extend passenger notifications and multilingual interfaces.
- Automate monitoring of model drift, uncertainty calibration and route-specific accuracy.

21. Conclusion

CHAKRAx6 proposes a dynamic ETA platform that combines live data fusion, section-wise machine learning, operational constraint modelling, explainability, continuous learning and digital-twin validation. The architecture is designed to work as a decision-support layer with existing railway systems, while providing more responsive ETA information to passengers and controllers.

The strongest final demonstration will be an end-to-end prototype showing a live or replayed train journey, event-triggered ETA changes, a P10-P90 confidence band, TreeSHAP explanation, CP-SAT delay propagation and measurable accuracy against the timetable baseline.

22. Submission Notes

- Use the same terminology as the PPT during presentation and viva.
- Add the final team website, GitHub and demo-video links when available.
- Convert the database model into a graphical ER diagram if required.
- Support the $\pm 5-8$ minute accuracy, 15-20% resource-utilization improvement and 13,000+ train scalability claims with test evidence before calling them achieved results.
- Keep mock/replay mode for safe prototype demonstration.